ENTERPRISE ARCHITECTURE BLUEPRINT

Cloud Accounting & Business Management Platform

(Codename "Ledger" - Zoho Books-class product)

VOLUME 7 OF 9

INFRASTRUCTURE & DEPLOYMENT ARCHITECTURE

CI/CD - Environments - Scaling - Observability - Disaster Recovery

Field	Value
Document Status	Draft - v0.1 - Phase 7 of 9
Prepared For	caldim
Prepared By	Claude (Enterprise Architecture Assistant)
Date	13 July 2026
Classification	Internal - Pre-Development Planning
Upstream Dependency	Vol. 2 Container Diagram; Vol. 3 Stack Decisions & Performance Targets; Vol. 4 Data volume/partitioning; Vol. 5 Integration Gateway network segment; Vol. 6 Section 7 Network Security, Section 11 Monitoring hooks
Downstream Volumes	Vol. 8 Testing & QA; Vol. 9 UI/UX & API Design

Table of Contents

1. Introduction & Scope

1.1 Purpose

This volume operationalizes every architectural decision made in Volumes 1 through 6: it selects the cloud provider and concrete managed services behind Volume 3's technology choices, defines environment topology and deployment architecture for Volume 2's containers, specifies the CI/CD pipeline (including the security and quality gates from Volume 6 Section 12 and the forthcoming Volume 8), and sets the scaling, observability, and disaster-recovery posture needed to actually run this platform in production.

1.2 Constraints Inherited From Earlier Volumes

- Vol. 3 Section 12 performance targets (P95 latency, 1,000+ then 10,000+ concurrent tenants) size everything in Section 7 of this volume.
- Vol. 4 Section 7's PgBouncer session-mode-pooling caveat for RLS compatibility is a hard constraint on Section 4's connection architecture.
- Vol. 6 Section 7.2's network segmentation (Integration Gateway restricted egress) is a hard constraint on Section 2's network topology.
- Vol. 6 Section 12's pre-release security test gates must be wired into Section 5's CI/CD pipeline, not treated as a separate manual step.

2. Environment Topology

2.1 Environment Tiers

Environment	Purpose	Data
Local/dev	Individual developer workstation; docker-compose spins up Postgres, Redis, Kafka locally	Synthetic seed data only
CI/ephemeral	Spun up per pull request for automated test suites (Vol.8), destroyed after the run	Synthetic seed data, reset every run
Staging	Pre-production, mirrors production topology at smaller scale; used for manual QA, demo, and the weekly DAST scan (Vol.6 Section 12)	Anonymized/synthetic data - never real tenant data
Production	Live tenant traffic	Real tenant data, full security controls from Vol.6 active

2.2 Network Topology

Network segmentation (implements Vol.6 Section 7.2)

```
flowchart TB
    Internet((Internet))
    WAF[WAF / Edge Protection]
    LB[Load Balancer]
    subgraph PublicSubnet[Public Subnet]
        Gateway[API Gateway]
    end
    subgraph AppSubnet[Private App Subnet]
        Core[Core Application - modular monolith]
        Workers[Background Workers]
        Payments[Payments Service]
    end
    subgraph RestrictedSubnet[Restricted Egress Subnet]
        IntGW[Integration Gateway - allowlisted outbound only]
    end
    subgraph DataSubnet[Private Data Subnet - no direct internet route]
        DB[(PostgreSQL - Primary + Replicas)]
        Cache[(Redis)]
        Kafka{{Kafka}}
        Search[(OpenSearch)]
    end

    Internet --> WAF --> LB --> Gateway
    Gateway --> Core
    Gateway --> Payments
    Gateway --> IntGW
    Core --> DB
    Core --> Cache
    Core --> Kafka
    Workers --> DB
    Workers --> Kafka
    IntGW -->|allowlisted domains only| External[[Bank/Payment/Tax/E-commerce APIs]]
    Kafka --> Search
```

3. Cloud Provider & Managed Services

3.1 Provider Decision

Field	Detail
Options Considered	AWS, Google Cloud Platform, Microsoft Azure
Decision	AWS, pending final confirmation (Vol.3 Section 13.2 open question) - this volume proceeds on that assumption and names AWS-equivalent services; the mapping to GCP/Azure equivalents is mechanical if the decision changes
Rationale	Broadest managed-service coverage for every component Vol. 3 selected (managed Postgres, Kafka, Redis, OpenSearch, object storage, KMS), largest talent pool, and the most mature multi-account/landing-zone patterns for the environment isolation in Section 2.1

3.2 Managed Service Mapping

Vol. 3 Component	AWS Managed Service	Notes
PostgreSQL (Vol.3 Section 2.2)	Amazon RDS for PostgreSQL (Multi-AZ)	Read replicas for Reporting query offload (Vol.3 Section 2.6 context)
Redis (Vol.3 Section 2.4)	Amazon ElastiCache for Redis	Cluster mode for horizontal scale beyond Release 1
Kafka (Vol.3 Section 2.5)	Amazon MSK (Managed Streaming for Kafka)	Avoids operating Kafka brokers directly, addressing Vol.3 Section 2.5's accepted ops-overhead consequence
OpenSearch (Vol.3 Section 2.6)	Amazon OpenSearch Service	Fed by the same Kafka topics via a managed connector or a lightweight consumer
Object storage (Vol.3 Section 2.7)	Amazon S3	Server-side encryption (Vol.6 Section 5.2)
KMS (Vol.6 Section 6.1)	AWS KMS	Backs envelope encryption for tokens/PII
Secrets manager (Vol.6 Section 6.2)	AWS Secrets Manager	Platform-level API keys and DB connection secrets
Container runtime (Section 4)	Amazon EKS (Kubernetes) or ECS Fargate	Decision detailed in Section 4.1

4. Deployment Architecture

4.1 Orchestration Decision

Field	Detail
Options Considered	Amazon ECS Fargate (serverless containers); Amazon EKS (managed Kubernetes); traditional EC2 autoscaling groups
Decision	ECS Fargate for Release 1, with a documented migration path to EKS if the team later needs Kubernetes-specific tooling (e.g. a service mesh for the extracted services from Vol.2 Section 2.1)
Rationale	Fargate removes node/cluster management overhead entirely, which matches the small-team, modular-monolith-first posture from Vol.2 Section 2.1 - the team should not be operating a Kubernetes control plane before it has more than a handful of independently deployable services (Payments, Integration Gateway) to justify it.

4.2 Deployment Units (maps to Vol. 2 Section 5 Container Diagram)

Deployment Unit	Fargate Service	Scaling Trigger
Core Application (modular monolith)	core-api	CPU/memory utilization + request queue depth
Payments Service (CTX-06)	payments-service	Request rate (independent from core, per Vol.2 Section 2.1 extraction rationale)
Integration Gateway (CTX-16)	integration-gateway	Queue depth (webhook/event backlog) rather than CPU, since it is I/O-bound
Background Workers (Vol.3 Section 9)	workers	Job queue depth (BullMQ/Redis queue length)
Web App / Portal App (static assets)	CDN + S3 origin (not a Fargate service)	N/A - served from CDN edge

4.3 Connection Pooling (PgBouncer)

Per Vol. 4 Section 7's caveat, PgBouncer runs in session mode (not transaction-pooling mode) for any connection path where RLS session variables (Vol. 3 Section 11.2) must persist across statements within a request - deployed as a sidecar or a dedicated pooling tier between the application services and RDS, sized to stay within RDS's max-connection limits as the Fargate service scales horizontally.

5. CI/CD Pipeline

5.1 Pipeline Stages

CI/CD pipeline (per pull request and per merge to main)

```
flowchart LR
    PR[Pull Request Opened] --> Lint[Lint + Module Boundary Check - Vol.3 Sec.4]
    Lint --> Unit[Unit Tests]
    Unit --> SAST[SAST + Dependency Scan - Vol.6 Sec.12]
    SAST --> IntTest[Integration Tests - ephemeral env, Section 2.1]
    IntTest --> TenantLeak[Cross-Tenant Leakage Suite - Vol.6 Sec.12]
    TenantLeak --> Merge{Merge to main}
    Merge --> Build[Build & Push Container Image]
    Build --> DeployStaging[Deploy to Staging]
    DeployStaging --> E2E[E2E Tests + Weekly DAST - Vol.6 Sec.12]
    E2E --> Approval{Manual Release Approval}
    Approval --> Canary[Canary Deploy to Production - 5% traffic]
    Canary --> Monitor[Automated Health Check Window]
    Monitor --> FullRollout[Full Production Rollout]
    Monitor -->|health check fails| Rollback[Automatic Rollback]
```

5.2 Deployment Strategy

Aspect	Approach
Rollout pattern	Canary (5% -> 25% -> 100%) with automated health-check gates between stages, per Section 5.1
Rollback	Automatic rollback on error-rate or latency-threshold breach during the canary window; manual rollback always available as a fallback
Database migrations	Applied as a separate pipeline step before the new application version receives traffic, following the expand/contract pattern (Vol.4 Section 9.2) so the previous version keeps working during rollout
Feature flags	New functionality ships behind a flag (Vol.3 Section 10) so deployment and feature release are decoupled - a bad deploy can be rolled back independently of a bad feature.

6. Infrastructure as Code

6.1 Tooling

Field	Detail
Decision	Terraform for all cloud infrastructure (networking, RDS, MSK, ElastiCache, ECS services, IAM), with modules organized per bounded-context deployment unit (Section 4.2) so a new extracted service can reuse an existing module pattern
State management	Remote state in S3 with DynamoDB locking, one state file per environment (Section 2.1) to prevent cross-environment blast radius from a single apply

6.2 Repository Structure

/infra folder (referenced from Vol.3 Section 3.2)

```
/infra
  /modules
    /networking        # VPC, subnets, security groups (Section 2.2)
    /database          # RDS, PgBouncer sidecar config
    /messaging         # MSK, topic provisioning (Vol.3 Section 8.1)
    /cache             # ElastiCache
    /search            # OpenSearch
    /compute           # ECS Fargate service definitions (Section 4.2)
    /security          # KMS, Secrets Manager, IAM roles (Vol.6 Section 6)
  /environments
    /dev
    /staging
    /production
```

7. Scaling Strategy

Sized against the Vol. 3 Section 12 performance targets: 1,000+ concurrent tenants at Release 1, 10,000+ at Release 3.

Component	Scaling Approach	Trigger
Core Application (Fargate)	Horizontal autoscaling, stateless service instances behind the load balancer	CPU > 60% or request queue depth threshold
PostgreSQL (RDS)	Vertical scaling of the primary for writes; horizontal read replicas for Reporting queries (Vol.3 Section 2.6)	Manual/scheduled resize initially; Release 3 evaluates the Vol.4 Section 11 tenant-isolation escape hatch for the largest accounts
Kafka (MSK)	Broker count and partition count sized to the topic design (Vol.3 Section 8.1), scaled ahead of projected event volume rather than reactively	Throughput monitoring against broker capacity
Redis (ElastiCache)	Cluster mode sharding once single-node capacity is insufficient	Memory utilization
Background Workers	Horizontal autoscaling independent of the core application	Job queue depth (Section 4.2)
OpenSearch	Horizontal node scaling for the Reporting/Analytics query load	Query latency / indexing lag

8. Observability

8.1 Three Pillars

Pillar	Tooling	Feeds From
Logs	Centralized log aggregation (e.g. CloudWatch Logs or an ELK/OpenSearch-based stack) ingesting the structured JSON logs defined in Vol.3 Section 6	Every service's stdout, shipped by a log agent
Metrics	Managed metrics/dashboard platform (e.g. CloudWatch + Grafana, or a vendor APM) tracking the Vol.3 Section 12 performance targets directly as dashboards	Application instrumentation + infrastructure-level metrics (RDS, MSK, ElastiCache)
Traces	Distributed tracing propagating the correlationId established at the Gateway (Vol.3 Section 6.1) across every module and, once extracted, every service	OpenTelemetry instrumentation in the NestJS application (Vol.3 Section 2.1)

8.2 Alerting (extends Vol. 6 Section 11)

Alert	Threshold	Severity
API P95 latency breach	Sustained above Vol.3 Section 12 target for 5 min	High
Event processing lag	Above Vol.3 Section 12 5s P95 target for 10 min	Medium
RLS bypass indicator (Vol.6 Section 8 layer 5)	Any occurrence	Critical - immediate page
Audit hash-chain break (Vol.6 Section 9.2)	Any occurrence	Critical - immediate page
Canary deploy health check failure	Error rate or latency regression during rollout (Section 5.2)	High - triggers automatic rollback
Dead-letter queue growth (Vol.3 Section 8.2)	Sustained growth over 15 min	Medium
Database connection pool exhaustion	PgBouncer (Section 4.3) at capacity	High

9. Disaster Recovery & Backup

9.1 Recovery Targets

Metric	Target	Scope
RPO (Recovery Point Objective)	< 5 minutes	Financial transactional data (ledger, receivables, payables) - achieved via RDS continuous backup / point-in-time recovery
RTO (Recovery Time Objective)	< 1 hour	Full platform restoration in a secondary AZ/region following a primary failure

9.2 Backup Strategy

Data Store	Backup Method	Retention
PostgreSQL (RDS)	Automated daily snapshots + continuous point-in-time recovery via transaction log shipping	Matches Vol.4 Section 10 retention policy for financial data; snapshots retained 35 days, PITR window 7 days
Kafka (MSK)	Multi-AZ replication (built into MSK); topic retention itself is the durability mechanism for recent events (Vol.3 Section 2.5), not a separate backup	Per-topic retention per Vol.3 Section 8.1 design
Redis (ElastiCache)	Not backed up - treated as ephemeral/rebuildable cache (Vol.3 Section 7), no durability guarantee needed	N/A
S3 (documents, exports)	Versioning enabled + cross-region replication for the production bucket	Matches Vol.4 Section 10 document retention policy
OpenSearch indices	Rebuildable from Kafka event history (Vol.3 Section 2.6) - not independently backed up, per Vol.4 Section 11's "derived data" classification	N/A - rebuild procedure documented in the ops runbook

9.3 Multi-AZ / Multi-Region Posture

Release 1 deploys Multi-AZ within a single AWS region (covers the RTO/RPO targets above against an availability-zone failure). Multi-region active-passive is a Release 3 consideration, driven by the same enterprise data-residency requirements flagged in Vol. 1 Section 13.2 and Vol. 4 Section 11.1's tenant-isolation escape hatch - both would likely be addressed together rather than as separate efforts.

10. Incident Management & On-Call

Extends Vol. 6 Section 11's security-specific monitoring hooks to general operational incident response.

Severity	Example	Response
SEV-1 (Critical)	Platform-wide outage, RLS bypass, audit hash-chain break, data loss	Immediate page, all-hands incident response, customer communication within 30 min
SEV-2 (High)	Single module degraded, elevated error rate, canary rollback triggered	Page on-call, response within 15 min, no mandatory customer communication unless prolonged
SEV-3 (Medium)	Non-critical background job failures, dead-letter queue growth	Alert during business hours, next-business-day response acceptable

Post-incident, a blameless postmortem is required for every SEV-1 and SEV-2, feeding back into this volume's alerting thresholds (Section 8.2) and Volume 8's regression test suite where the incident reveals a testing gap.

11. Cost Considerations (Summary)

Detailed cost modeling is outside this architecture blueprint's scope, but the following architectural choices were made with cost-consciousness in mind and should inform budget planning:

- Fargate over self-managed Kubernetes (Section 4.1) trades some raw compute-cost efficiency for materially lower operational/engineering cost at Release 1 scale.
- Read replicas and OpenSearch (Section 7) are scaled to Reporting's actual query load rather than provisioned for peak-guess capacity, reviewed quarterly against real usage.
- The Release 3 multi-region option (Section 9.3) roughly doubles infrastructure cost for the tenants that need it - this is why it is scoped as an opt-in Enterprise capability (Vol.1 Section 9) rather than a platform-wide default.
- Cold-storage tiering for aged partitions (Vol.4 Section 6, Section 10) keeps long-tail compliance retention from inflating primary-database storage costs.

12. Environment Promotion & Release Management

Promotion flow

```
flowchart LR
    Dev[Local/Dev] --> CI[CI - ephemeral env]
    CI --> Staging[Staging]
    Staging -->|manual QA + automated E2E, Vol.8| Approval{Release Approval}
    Approval --> Prod[Production - Canary Rollout, Section 5.2]
```

Release cadence targets weekly deploys to staging and a controlled, on-demand production release once the canary health checks (Section 5.2) pass - not a fixed release train - so a critical fix does not have to wait for a scheduled window, while routine feature work batches naturally through the weekly staging cycle.

13. Traceability & Next Phase

13.1 Traceability to Volumes 1-6

Vol. 7 Artifact	Traces To
Section 2.2 Network Topology	Vol. 6 Section 7.2 Network Segmentation
Section 3.2 Managed Service Mapping	Vol. 3 Section 2 Technology Stack Selection
Section 4.2 Deployment Units	Vol. 2 Section 5 Container Diagram
Section 4.3 PgBouncer	Vol. 4 Section 7 RLS connection-pooling caveat
Section 5.1 CI/CD Gates	Vol. 6 Section 12 Security Testing Requirements
Section 7 Scaling	Vol. 3 Section 12 Performance & Scalability Targets
Section 9 DR/Backup	Vol. 4 Section 10 Data Retention & Archival

13.2 Open Questions for Sign-off

1. Confirm AWS as the cloud provider (Section 3.1) so the managed-service mapping and Terraform modules (Section 6) can be finalized rather than kept provider-agnostic.

2. Confirm whether a formal RTO/RPO commitment (Section 9.1) needs to appear in customer-facing SLAs, which would affect how aggressively Release 1 must invest in Multi-AZ resilience versus deferring some of it.

3. Confirm on-call staffing model (Section 10) - whether the team has dedicated on-call capacity at launch or whether SEV-1 response in the early weeks relies on the founding engineering team directly.

13.3 Next Phase

On approval, Phase 8 begins: Testing & QA Specification (Volume 8) - the full test strategy (unit, integration, E2E, load, security) and CI gate definitions that this volume's pipeline (Section 5) assumes, followed by Phase 9: UI/UX Design Specification & API Design Document (Volume 9), which will complete the nine-volume blueprint.